Python Avancée

Data Science et Sécurité des Systèmes d'Information

Système de Gestion des Emplois du Temps pour une Faculté

Réalisé par :

Abderrahmane Mazygh

Résumé

Ce document présente la synthèse d'un mini-projet réalisé dans le cadre du module Python avancée. Ce travail s'inscrit dans le projet de fin Module intitulé : « Système de Gestion des emplois du temps pour une Faculté ».

L'objectif de ce projet est de concevoir un système de gestion des emplois du temps pour une faculté afin d'optimiser la planification des cours, TD et TP, tout en assurant une répartition équitable de la charge horaire des enseignants et une meilleure utilisation des salles. Ce système doit offrir une interface simple et efficace pour les administrateurs, enseignants et étudiants.

Le projet vise à automatiser la gestion des emplois du temps, incluant la création et gestion des départements, filières, utilisateurs (administrateurs, enseignants, étudiants), ainsi que la planification des cours avec détection des conflits horaires. Il permet l'attribution des salles selon leur disponibilité et capacité, le calcul des charges horaires des enseignants, et la génération/exportation des emplois du temps (PDF, Excel, ICS). Un système de notifications informe les utilisateurs des modifications, avec une authentification sécurisée et une gestion des rôles.

Le projet a été mené selon la méthodologie Scrum, avec des sprints couvrant l'étude fonctionnelle, la conception et l'implémentation. Les technologies utilisées incluent Python (avec Django), MySQL pour la base de données, et HTML/CSS/JavaScript avec Bootstrap pour l'interface web. Des bibliothèques comme Pandas, Matplotlib, ReportLab et openpyxl ont été employées pour la gestion des données, la visualisation et l'exportation. Des concepts avancés tels que la programmation orientée objet (POO), les décorateurs, les générateurs, les itérateurs, et des design patterns (Singleton, Factory) ont été mis en œuvre pour assurer modularité et performance. La sécurité est renforcée par le chiffrement des mots de passe et la gestion des permiss

Abstract

"This document presents the summary of a mini-project carried out as part of the *Advanced Python* module. This work is part of the end-of-module project titled: "*Timetable Management System for a Faculty."*

The objective of this project is to design a timetable management system for a faculty to optimize the scheduling of lectures, tutorials, and practical sessions, while ensuring a fair distribution of teachers' workload and better utilization of classrooms. The system must provide a simple and efficient interface for administrators, teachers, and students.

The project aims to automate timetable management, including the creation and management of departments, programs, users (administrators, teachers, students), and course scheduling with conflict detection. It enables room allocation based on availability and capacity, calculation of teachers' workload, and generation/export of timetables (PDF, Excel, ICS). A notification system informs users of changes, with secure authentication and role management.

The project was conducted using the Scrum methodology, with sprints covering functional analysis, design, and implementation. The technologies used include Python (with Django) MySQL for the database, and HTML/CSS/JavaScript with Bootstrap for the web interface.

Résumé

Le présent travail intitulé «Système de Gestion des Emploi du Temps pour une faculté» s'inscrit dans le cadre de projet de fin module Python avancée. Notre projet a pour objectif de mettre en place un Système de Gestion des emplois du temps pour une faculté afin d'optimiser la planification des cours, TD et TP, tout en assurant une répartition équitable de la charge horaire des enseignants et une meilleure utilisation des salles.

Il s'agit d'une application qui regroupe les activités de gestion d'emploi du temps dans les établissements universitaires. Elle se consacre à la conception et à l'implémentation d'un outil capable d'automatiser, ainsi de faciliter et d'optimiser cette tâche.

Cet outil permet la manipulation des données telle que l'ajout, la suppression, la modification, et la sélection des documents intégrés dans l'application Gestion d'Emploi du Temps.

De nos jours, les organisations connaissent de perpétuels changements (restructurations, évolutions de métiers, mutations technologiques…). D'autre part, des nombreuses universités migrent de plus en plus vers les systèmes informatiques centralisés au détriment de leurs systèmes de gestion de bases de données traditionnels. Le besoin en matière d'accès aux informations est devenu pressant pour permettre aux utilisateurs de gérer et de suivre tout ce qui rapporte à l'université et surtout la gestion des emplois du temps.

Ces systèmes d'informations se révèlent un vecteur de performance essentiel pour la productivité de l'université puisqu'ils sont des outils logiciels de pilotage de l'activité départementale et administrative, permettant à une université, et en particulier l'FPBM, de gérer et d'optimiser cette activité.

En effet, les activités administratives au sein d'une université sont diverses (gestion des emplois de temps, gestion des absences des étudiants, gestion de notes, gestion des salles,…) et gérés par plusieurs employés (chef des départements, enseignants, employé administratif…). Donc, il important de développer un système informatique centralisé pour que l'information soit accessible à tous.

Sur le marché de logiciels, ces genres de systèmes informatiques existent d'une manière indépendante, c'est-à-dire que pour chaque activité au sein de l'université un logiciel spécifique le gère. Ce qui amène à une redondance des données, difficultés à manipuler les informations et un travail répétitif qui mène à une perte de temps.

D'où l'importance de développer un système informatique qui assemble toutes les activités d'une université dans une seule plateforme. Plateforme centralisée, accessible à distance et qui rend l'information universitaire disponible.

C'est dans ce cadre que s'intègre notre projet de fin module qui s'est déroulé au sein de La faculté polydisciplinaire de Béni Mellal.

De ce fait, notre objective visait à mettre en œuvre une solution logicielle intégrée et hautement paramétrable pour la gestion d'emploi du temps qui souscrit les différentes opérations demandées par l'Université.
Notre solution doit être puissante et souple à la fois, permettant d'organiser le travail.

Afin d'illustrer la démarche de notre travail, nous présentons dans ce qui suit l'organisation générale de ce rapport. En effet, le présent rapport s'articule autour de trois chapitres.
Le premier chapitre intitulé " Présentation et analyse du problème ", est réservé à l'étude de l'existant en cherchant les objectifs à développer, décrit les besoins de notre projet, qu'ils soient fonctionnels ou non fonctionnels.

Dans le deuxième chapitre, nous allons présenter la conception de l'application.

Dans le troisième chapitre, nous allons présenter la phase de l'implémentation de l'application en justifiant les différents choix techniques et technologiques utilisés au cours de cette phase avant de projeter quelques captures écran de l'application finale réalisée.
Nous terminons ce rapport par une conclusion générale.

Chapitre 1

Présentation et analyse du problème

Introduction

La mise en place d'un logiciel, passe essentiellement par une étude de l'existant, qui offre la connaissance précise et rigoureuse du contexte sur lequel nous opérons et guide ainsi à une décision pour démarrer ou simplement marqueter un tel projet. C'est le sujet de ce chapitre qui présentera les principaux aspects techniques et normatifs sur lesquels reposera notre travail.

I. Présentation

1. Présentation du projet

2.1 Cadre du projet

Ce projet s'inscrit dans le cadre de la réalisation de projet de fin Module Python avancée au sein de l'FPBM et intitulé « Système de Gestion des Emplois du Temps ». Le but de ce travail est de développer un logiciel qui permet de garantir plus de fonctionnalités spécifiquement dédié aux établissements universitaires pour fournissant un moyen de recherche et de gestion plus efficace.

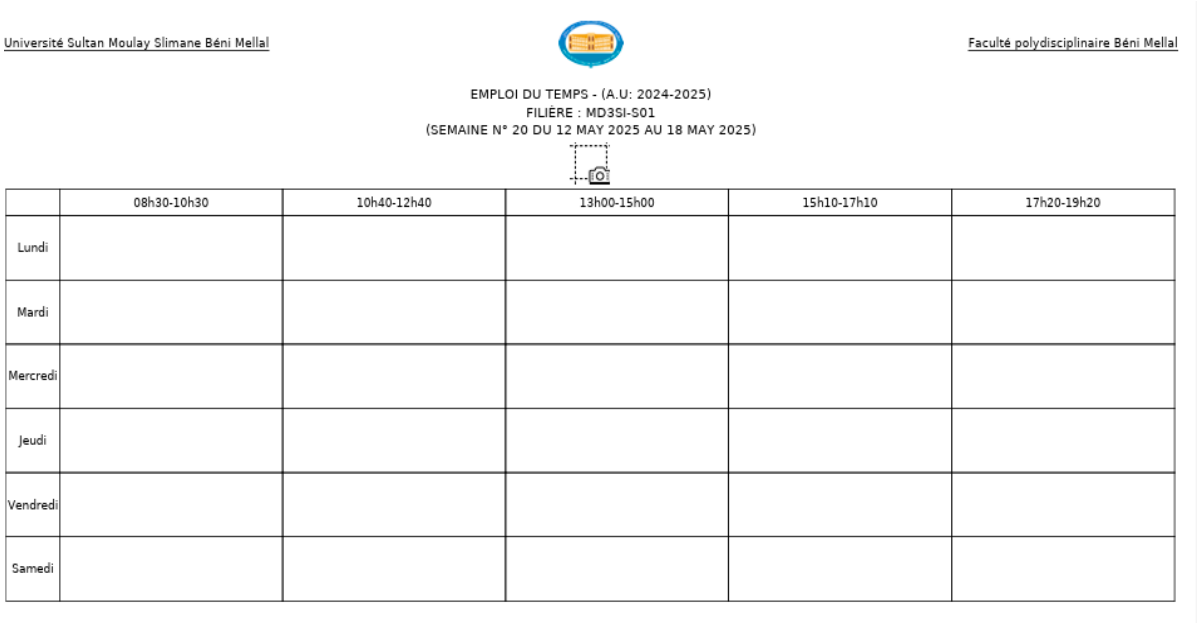
2.2 Introduction à la Gestion d'Emplois du Temps

Chaque année, les responsables pédagogiques des universités et des Instituts Supérieurs des Etudes Technologiques (FP) ont pour mission d'organiser les emplois du temps des différentes formations ou filières en essayant, au mieux, de satisfaire les contraintes «humaines» des enseignants et des étudiants, les contraintes pédagogiques imposées par la progression des enseignements et en tenant compte des contraintes « physiques » liées aux ressources matérielles (les salles, les départements , etc.).

On constate, en effet, que lors de la création d'un emploi du temps, les responsables partent généralement d'un emploi du temps existant auxquels ils apportent des modifications. La méthode ne consiste donc pas à créer mais à adapter un emploi du temps. D'où la nécessité de disposer d'outils interactifs facilitant ces adaptations. Contrairement aux outils de génération automatique, les outils recherchés sont « centrés utilisateur ». Il convient de revenir à des outils totalement sous le contrôle de l'utilisateur, adaptés aux démarches de travail selon différents points de vue liés aussi bien à la conception, qu'à la visualisation (consultation et recherche) et la modification des emplois du temps. Dans cette optique, l'utilisateur redevient l'acteur central de la résolution du problème, et les outils s'articulent autour de son activité. Dans la suite, nous parlerons donc des acteurs (utilisateurs) de l'application.

L'outil recherché doit donc avant tout être interactif, souple, ouvert et doit présenter des qualités ergonomiques. Ces raisons nous ont motivés à réfléchir sur la conception d'un outil d'aide à la manipulation des emplois du temps suffisamment ouvert pour permettre de nombreuses extensions. Cette réflexion a abouti à la conception notre application de Gestion d'Emploi du Temps.

Cette application permet de gérer plusieurs modules. Parmi ces modules nous pouvons citer la gestion d'emploi du temps, des notes, des absences, des enseignants, des étudiants,...



Université Sultan Moulay Slimane Béni Mellal

Faculté polydisciplinaire Béni Mellal

EMPLOI DU TEMPS - (A.U: 2024-2025)
FILIÈRE : MD3SI-S01
(SEMAINE N° 20 DU 12 MAY 2025 AU 18 MAY 2025)

	08h30-10h30	10h40-12h40	13h00-15h00	15h10-17h10	17h20-19h20
Lundi					
Mardi					
Mercredi					
Jeudi					
Vendredi					
Samedi					

Figure 1 : gestion classique d'emploi du temps

II. Problématique

Dont le but de satisfaire les besoins de l'FP en développant un logiciel de qualité, Ce logiciel permettra de dégager les inconvénients du travail manuel et la propagation de temps.

1. Critique de l'existant

Actuellement, la gestion des emplois de temps à l'FP de BM se fait manuellement.

Ensuite, les emplois de temps seront enregistrés dans un document Word. Ce qui rend ce travail pénible et difficile à maintenir, parmi les difficultés :

- La perte de temps lors de la création manuelle des emplois de temps.
- L'utilisation des méthodes classique.

- Génération des emplois de temps des groupes, des salles, et des enseignants se fait d'une manière manuelle (n'est pas automatique).
- Plusieurs difficultés dans l'opération de recherche des documents.

2. Les solutions proposées

Pour résoudre les problèmes signalés précédemment, nous proposons de:

- Développer une application contenant tous les modules de gestion d'emploi du temps.
- Mettre l'application dans le site Web de l'FP pour être accessible aux enseignants et aux étudiants à distance.
-

3. Planification et conduite de projet

3.1 Méthodologie de travail

Traditionnellement, la modélisation des systèmes se base sur les hypothèses suivantes :

- Les fonctionnalités d'un système sont stables, elles n'évoluent pas avec le temps.
- Les besoins relatifs à un système sont donnés au départ.
- La validation des besoins peut se faire en référence aux fonctionnalités du système.

Aujourd'hui il est clair, que ces hypothèses ne sont plus valides, ainsi une séparation entre l'aspect fonctionnel et l'aspect technique lors du développement est devenue nécessaire pour livrer un produit qui respecte les besoins des utilisateurs tout en s'adaptant aux futurs changements, nous avons opté pour une méthode agile.

2.1 Principes de la méthode Agile

Les méthodes agiles partent du principe que spécifier et planifier dans les détails l'intégralité d'un produit avant de le développer (approche prédictive) est contre-productif. Cela revient à planifier dans les détails de trajet en voiture par les petites routes.

Spécifiant chaque ville et villages traversés, l'heure de passage associée, chaque rue empruntée dans les agglomérations, litres d'essence consommés, kilomètres parcourus, etc. Les imprévus ne manqueront pas d'arriver : embouteillages, déviations, travaux, sens de

circulation inversés, voire la panne, etc. Rendant votre planification et vos spécifications très vite obsolètes. Combien de temps aurez-vous passé à planifier cet itinéraire, comment réagirez-vous face à vos frustrations de ne pas pouvoir appliquer votre plan à la lettre ?

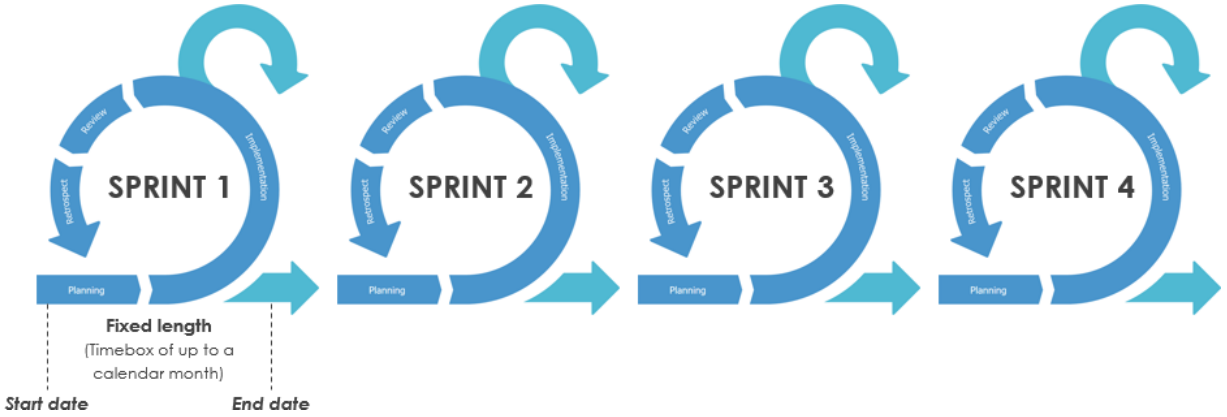
L'idée consiste à se fixer un premier objectif à courts termes (une grande ville par exemple) et se lancer sur la route sans tarder. Une fois ce premier objectif atteint, on marque une courte pause et on adapte son itinéraire en fonction de la situation du moment. Et ainsi de suite jusqu'à atteindre la destination finale. On parle donc d'une approche empirique. Dans le cadre d'un projet de développement logiciel, le client élabore sa vision du produit à réaliser et liste les fonctionnalités ou exigences de ce dernier. Il soumet cette liste à l'équipe de développement, communique directement avec elle (plutôt que par papier) qui estime le coût de chaque élément de la liste. On peut ainsi se faire une idée approximative du budget global.

Dans ce projet, la méthode la plus adaptée est la méthode agile vu qu'elle s'adapte au fur et à mesure avec les besoins de notre mini projet et les technologies utilisés.

Selon le besoin exprimé, nous avons remarqué qu'il peut être subdivisé en modules quasiment indépendants avec plusieurs tâches de courte à moyenne durée.

3.2 Méthode SCRUM

Afin de répondre aux attentes du client en termes de souplesse et réactivité dans la démarche de mise en œuvre, le projet repose sur la méthode SCRUM. Cette méthode s'appuie sur l'organisation de développement de produits complexes, il est défini comme un cadre de travail holistique itératif qui se concentre sur les buts communs en livrant de manière productive et créative des produits de la plus grande valeur possible.



4.1 Les caractéristiques de SCRUM

Scrum permet de mettre en œuvre un pilotage du projet par la valeur livrée au fil des itérations appelées sprints. Ses principales caractéristiques sont les suivantes :

- **Priorisation** : Les fonctionnalités les plus importantes seront réalisées en premier.
- **Itératif** : L'équipe développe ces fonctionnalités par itérations courtes et de durée fixe (généralement entre 2 et 5 semaines), appelées les Sprints. Les fonctionnalités réalisées dans un sprint, listées dans le Product Backlog, sont sélectionnées en accord avec le Product Owner lors du sprint planning avant le début de chaque sprint.
- **Incrémental** : Chaque sprint aboutit à un incrément du produit opérationnel (potentiellement livrable). A la fin du sprint, lors du Sprint Review, l'équipe fait une démonstration du produit avec les fonctionnalités réalisées.
- **Adaptatif** (réponse au changement) : Entre chaque sprint, les priorités métier peuvent changer et les évolutions sont les bienvenues. Il suffit de mettre à jour le Product Backlog.
- **Productif** : Une fois qu'un sprint est commencé, l'équipe se concentre sur le développement des fonctionnalités sélectionnées.
- **Amélioration continue** : Tous les jours, l'équipe fait un point de synchronisation et d'avancement (15mn).

5.1 Principes et avantages

Quant aux principes de la méthode SCRUM et ce qu'ils nous ont procurés comme avantages tout au long du projet.

- SCRUM étant une méthode itérative et incrémentale, elle nous a permis d'éviter « l'effet tunnel », c'est-à-dire le fait de ne voir le résultat qu'à la livraison finale et rien ou presque rien pendant toute la phase de développement. Au-delà du fait de voir le résultat de notre travail rapidement, cette facette du processus SCRUM nous a également permis de repérer, corriger et modifier quand cela a été nécessaire assez tôt dans le projet.
- La flexibilité de cette méthode due à la composition séquentielle du contenu des sprints nous a permis d'ajouter des modifications et des fonctionnalités qui n'étaient pas prévues au départ.

- SCRUM augmente et facilite la communication également, en travaillant dans la même salle de développement, ou en étant connecté avec différents moyens de communication. Les échanges ont pour objectif de supprimer les obstacles le plus tôt possible.
- Augmentation de la productivité : en supprimant certaines « contraintes » des méthodes classiques comme la documentation ou la formalisation exagérées, SCRUM nous a également permis d'augmenter notre productivité.

6.1 L'équipe SCRUM

Dans une équipe Scrum, il y a Trois rôles : le Product Owner, l'équipe de développement et le Scrum Master :

- **Product Owner** : dans la majorité des projets, le responsable produit (Product Owner) est le responsable de l'équipe projet client. C'est lui qui va définir et prioriser la liste des fonctionnalités du produit et choisir la date et le contenu de chaque sprint sur la base des valeurs (charges) qui lui sont communiquées par l'équipe,

- **Scrum Master** : est responsable de la compréhension, de l'adhésion et de la mise en œuvre de la méthode Scrum qu'il maîtrise parfaitement. Il veille à ce que les principes et les valeurs de la méthodologie sont respectés. C'est un facilitateur qui aide à améliorer la communication au sein de l'équipe et cherche à maximiser la productivité et le savoir-faire de celle-ci. Il est considéré comme le coach de l'équipe de développement.

- **Équipe de développeurs** : l'équipe s'organise elle-même et elle reste inchangée pendant toute la durée d'un sprint. Elle est chargée de transformer les besoins définis par le Product Owner en fonctionnalités utilisables. Elle est pluridisciplinaire et possède toutes les compétences nécessaires pour réaliser le projet, sans faire appel à des prestations externes. Parmi ses membres, on trouve un architecte, un développeur, un testeur, etc. La taille idéale de l'équipe de développement est de 3 à 10 personnes. Il n'y a pas de notion de hiérarchie, toutes les décisions sont prises ensemble.

7.1 Les cérémonies SCRUM

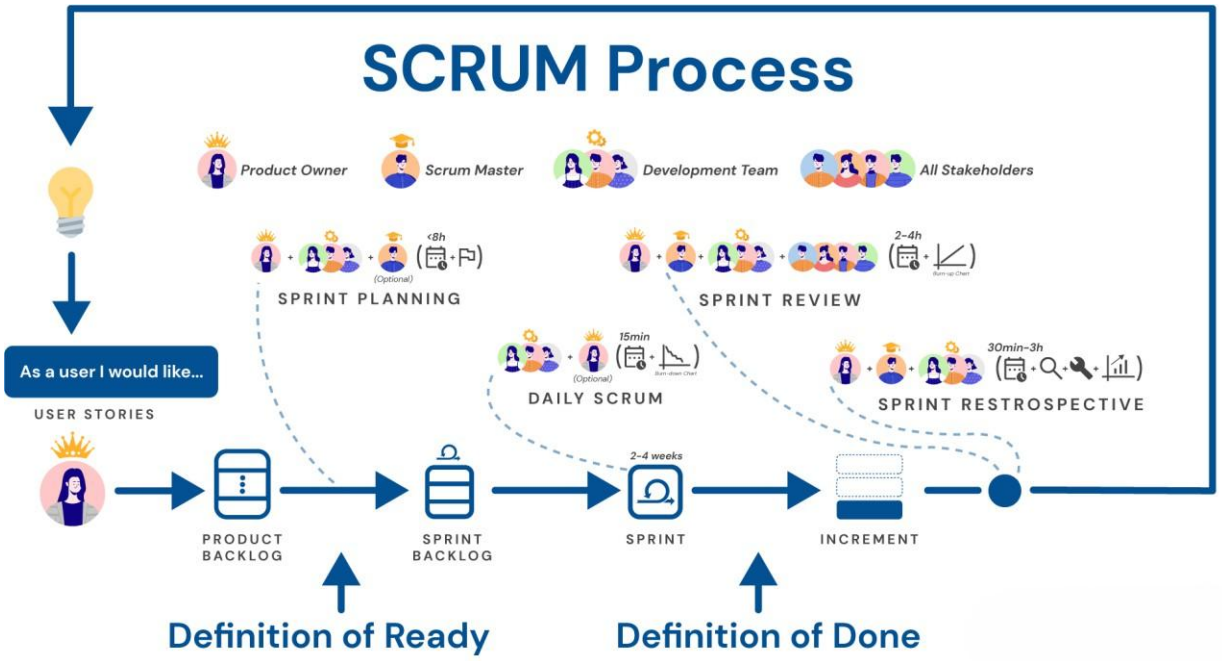


Figure 2: Processus détaillé de SCRUM

- **Sprint planning** : Est un événement dans lequel une équipe agile détermine les objectifs du sprint et la quantité de backlog de l'équipe à laquelle elle peut s'engager lors d'un sprint à venir. La capacité de l'équipe détermine le nombre de story point et de facilitateurs sélectionnés.

- **Daily Meeting** : Comme son nom l'indique, cette réunion a lieu tous les jours. Sa durée maximale est de 15 minutes. Son objectif est de permettre à l'équipe agile d'évoquer l'avancement des taches, les blocages éventuels et l'humeur pour chaque membre.

- **Sprint Review** : Est un événement basé sur la cadence à la fin de chaque itération dans lequel l'équipe examine les résultats de l'incrément précédent et ajuste le backlog de l'équipe en fonction des commentaires.

- **Rétrospective** : Il s'agit d'un événement organisé à la fin de l'itération pour que l'équipe agile passe en revue ses pratiques et identifie les moyens de s'améliorer. La rétrospective est basée sur les informations qualitatives et quantitatives présentées lors de la revue d'itération.

- **Grooming** : Est un événement organisé une ou deux fois au cours de l'itération pour

affiner, examiner et estimer les futures histoires et catalyseurs dans le backlog de l'équipe.

8.1 Planification de projet

La planification du projet est une étape primordiale et nécessaire à la conduite d'un projet informatique. En effet, la planification de projet correspond à l'organisation des tâches à réaliser sur une période donnée.

L'objectif de la planification est de déterminer le coût, les ressources mobilisées et la meilleure manière d'ordonnancer toutes les tâches à effectuer.

Les réunions tenues avec l'encadrant nous ont permis de planifier le déroulement du projet et de diviser les tâches, et attribuer un volume horaire pour chacune des phases suivantes du projet :

- ⇨ Documentation et collecte des informations nécessaires
- ⇨ Formation dans les outils utilisés ;
- ⇨ Monté en compétence dans le projet ;
- ⇨ Analyse et étude de faisabilité ;
- ⇨ Conception de la solution ;
- ⇨ Préparation de l'environnement de développement ;
- ⇨ Développement de la fonctionnalité

Par ailleurs, nous avons également utilisé un outil de tableau Kanban afin d'assurer un suivi optimal en temps réel des tâches en cours de réalisation en les subdivisant en les catégories suivantes :

- **Ready / to correct**: indiquant les tâches prêtes à se réaliser.
- **WIP** : Work in Progress' indiquant ainsi les tâches en cours de réalisation.
- **To Review** : Indiquant les tâches sous la revue des responsables techniques.
- **To Test :** Indiquant les tâches sous les tests.
- **Done :** Indiquant les tâches considérées comme achevées.

III. Analyse de besoins et spécification

1. Spécification des besoins fonctionnels

Dans cette section nous allons étudier les besoins fonctionnels, non fonctionnel et technique du notre système.

- L'administrateur peut :

  Gérer les emplois du temps, les groupes, les absences, les étudiants, les enseignants, les matières, les locaux.
- L'enseignant peut :

  Consulter les enseignants, les étudiants, les matières, les emplois de temps...
- L'étudiant peut :

  Consulter les heures plus et moins chargés ,les emplois de temps .

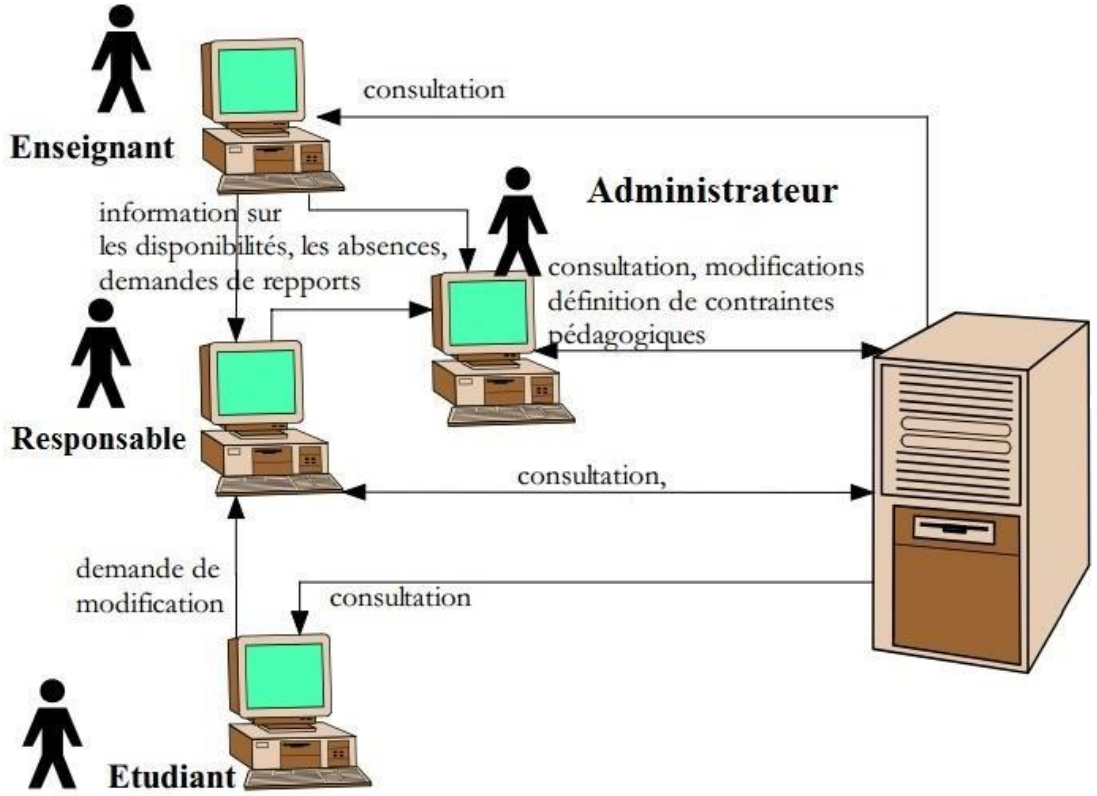


Figure 3 : Architecture technique proposée pour le problème des emplois du temps

2. Spécification des besoins non fonctionnels

Les besoins non fonctionnels décrivent toutes les contraintes auxquelles est soumis le logiciel pour sa réalisation et son bon fonctionnement :

- **BNF 1** Ergonomie et souplesse

L'application doit offrir une interface conviviale et ergonomique exploitable par l'utilisateur en envisageant toutes les interactions possibles à l'écran du support tenu.

- **BNF 2** Rapidité

L'application doit optimiser les traitements pour avoir un temps de génération de schéma raisonnable.

- **BNF 3** Efficacité

L'application doit être fonctionnelle indépendamment de toutes circonstances pouvant entourer l'utilisateur.

- **BNF 4** Maintenabilité et scalabilité

Le code de l'application doit être lisible et compréhensible afin d'assurer son état évolutif et extensible par rapport aux besoins.

Conclusion

Une étude des applications existantes concurrentes s'est avérée indispensable pour notre étude théorique du sujet. Lors de cette étude nous avons recouru à une comparaison entre quelques logiciels de gestion de note pour les universités présents en identifiant leurs limites et les objectifs de notre logiciel et ses apports.

Au niveau du chapitre suivant nous allons entamer notre étude conceptuelle.

Chapitre 2

CONCEPTION DÉTAILLÉE DE LA SOLUTION

Introduction

Après avoir défini la spécification du projet, la phase de conception vient pour mieux l'éclaircir. En effet, une phase de conception a pour objectif de permettre de formaliser les étapes préliminaires du développement d'un système afin de rendre ce dernier plus fidèle aux besoins des acteurs.

Elle permet de décrire de manière non ambigüe le fonctionnement désiré du système afin d'en faciliter la réalisation et la maintenance. Dans une première partie nous présenterons une conception globale de notre projet. Ensuite, dans la deuxième partie nous détaillerons la conception en utilisant les diagrammes UML appropriés.

I. Technique de modélisation

1. Choix méthodologique et conceptuel

Nous avons utilisé, pour la spécification et la conception de ce travail, la méthodologie UML (Unified Modeling Langage), qui est un langage de modélisation qui permet de décrire les besoins, de documenter les systèmes et d'esquisser les architectures logicielles. UML s'articule autour de neuf diagrammes différents, chacun d'eux étant dédié à la présentation des concepts particuliers d'un système logiciel.

Toutefois, nous ne présenterons que quelques diagrammes que nous avons jugés utiles et suffisants pour comprendre le projet à savoir les diagrammes de cas d'utilisations, les diagrammes de classes et les diagrammes de séquences.

2. Diagramme de cas d'utilisation

Les cas d'utilisation permettent de traduire les spécifications fonctionnelles d'utilisation du système. Ils décrivent sous la forme d'actions et de réactions le comportement du système du point de vue d'un utilisateur et représentent un ensemble de séquences d'action réalisées par le système et produisant un résultat observable pour un acteur particulier.

Les acteurs de l'application

Un acteur est une entité externe qui agit sur le système. Il peut consulter ou gérer l'état du système selon son rôle.

Notre application a comme des acteurs :

➢ **Administrateur :** L’administrateur est le principal gestionnaire du système. Il s’authentifie pour accéder à toutes les fonctionnalités de gestion, telles que la création et la modification des cours, des professeurs, des collaborateurs, des étudiants, des filières, des salles, et des emplois du temps. Il peut également modifier son propre profil et envoyer des e-mails aux autres utilisateurs. Son rôle est essentiel pour maintenir l'organisation et la mise à jour du système.

➢ **Enseignant :** L’étudiant se connecte au système avec une authentification dédiée. Il a accès à son tableau de bord où il peut consulter son emploi du temps et voir les cours auxquels il est inscrit. Il peut également consulter et modifier son profil pour garder ses informations à jour. Cela lui permet de suivre efficacement ses activités académiques.

➢ **Etudiant :** L’utilisateur représente toute personne qui n’est pas encore enregistrée dans le système. Il peut créer un compte pour devenir étudiant ou professeur, ou bien demander la réinitialisation de son mot de passe en cas d’oubli. C’est par cette interface que les nouveaux utilisateurs accèdent au système.

2.1 Diagramme des cas d’utilisation :

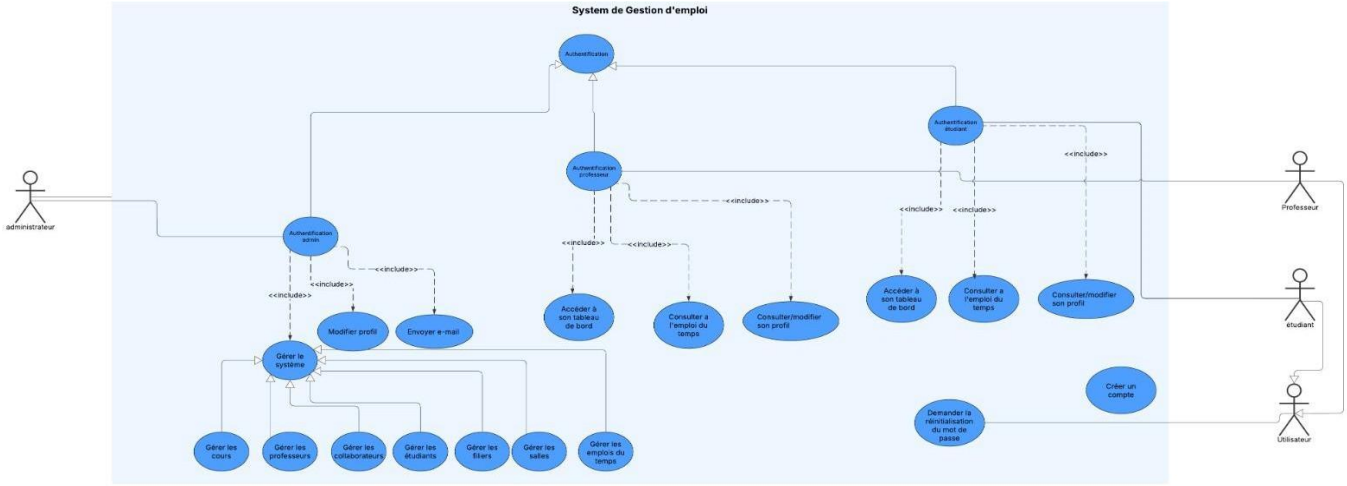


Figure 4 : Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présenté permet de visualiser le fonctionnement global du système de gestion d'emploi du temps en représentant les différents types d'utilisateurs (acteurs) ainsi que les actions qu'ils peuvent effectuer (cas d'utilisation).

Chaque acteur (administrateur, professeur, étudiant ou utilisateur générique) interagit avec le système à travers une série de fonctionnalités spécifiques.

Par exemple, **l'administrateur**, après authentification, peut accéder à une interface de gestion lui permettant de contrôler l'ensemble des éléments du système tels que les cours, les salles, les professeurs, les filières, etc.

Les professeurs et les étudiants, eux, peuvent consulter leur emploi du temps, accéder à leur tableau de bord et gérer leur profil.

Le diagramme utilise également des relations <<include>> pour indiquer les fonctionnalités communes entre plusieurs cas, comme la modification du profil ou l'authentification.

Ce schéma permet ainsi d'avoir une vue d'ensemble claire sur les interactions possibles avec le système, tout en mettant en évidence les responsabilités et privilèges de chaque utilisateur.

3. Diagramme de séquence

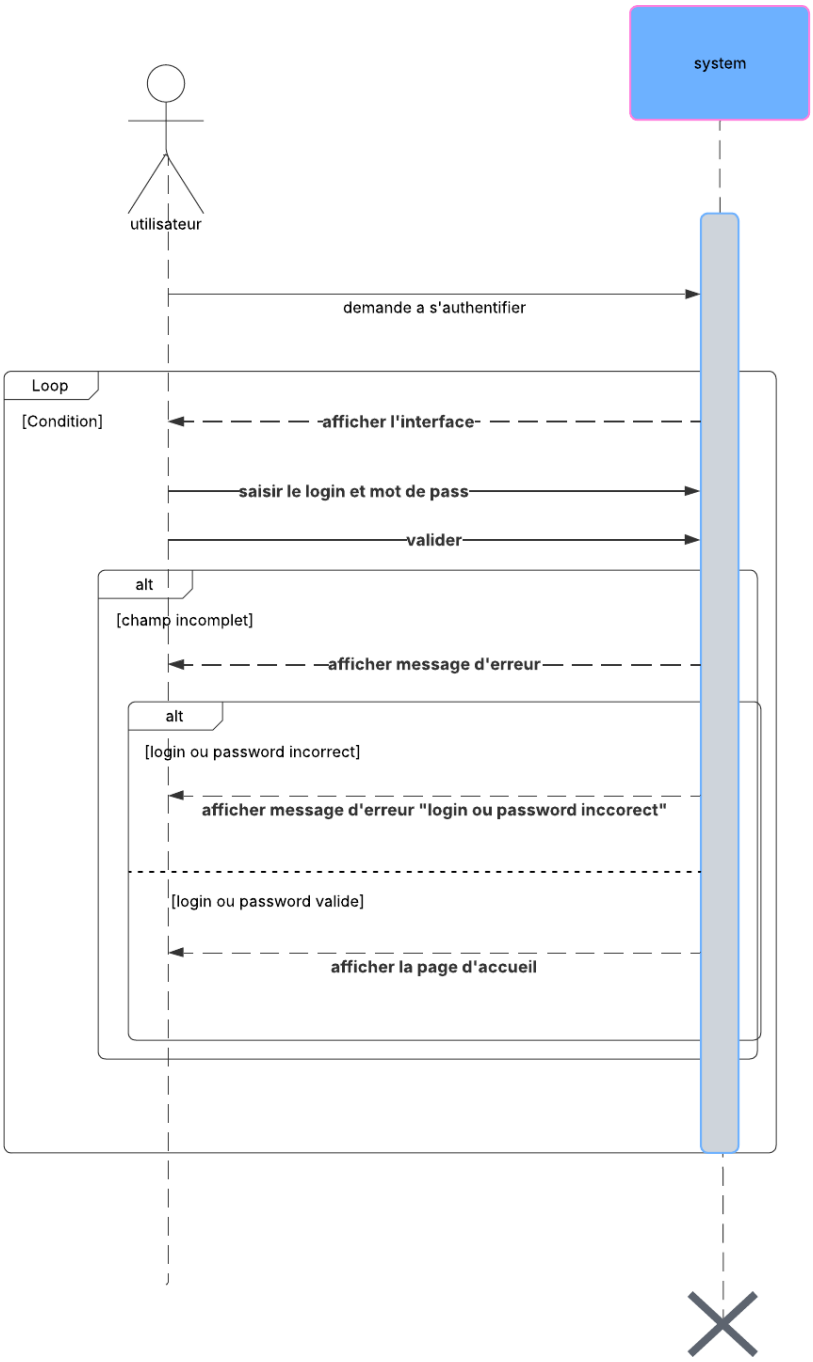


Figure 5 : Diagramme de séquence(d’authentification)

Ce diagramme de séquence illustre le processus d'authentification d'un utilisateur dans le système.

Le scénario débute lorsque l’utilisateur demande à s’authentifier. Le système lui affiche alors l’interface de connexion. L’utilisateur saisit son login et son mot de passe, puis valide sa demande.

À ce stade, plusieurs cas peuvent se produire, représentés par des blocs alternatifs (alt). Si les champs requis ne sont pas complétés, un message d’erreur est affiché pour demander à l’utilisateur de remplir tous les champs.

Si les champs sont renseignés mais que les identifiants sont incorrects, un message d’erreur indiquant « login ou password incorrect » est renvoyé.

En revanche, si les informations saisies sont valides, le système authentifie l’utilisateur et affiche la page d’accueil.

Le diagramme utilise également une boucle (loop) pour représenter les tentatives multiples que l'utilisateur peut faire jusqu'à une authentification réussie.

Ce diagramme permet ainsi de mieux comprendre la logique d’interaction entre l'utilisateur et le système lors du processus de connexion.

4. Diagrammes d’activité

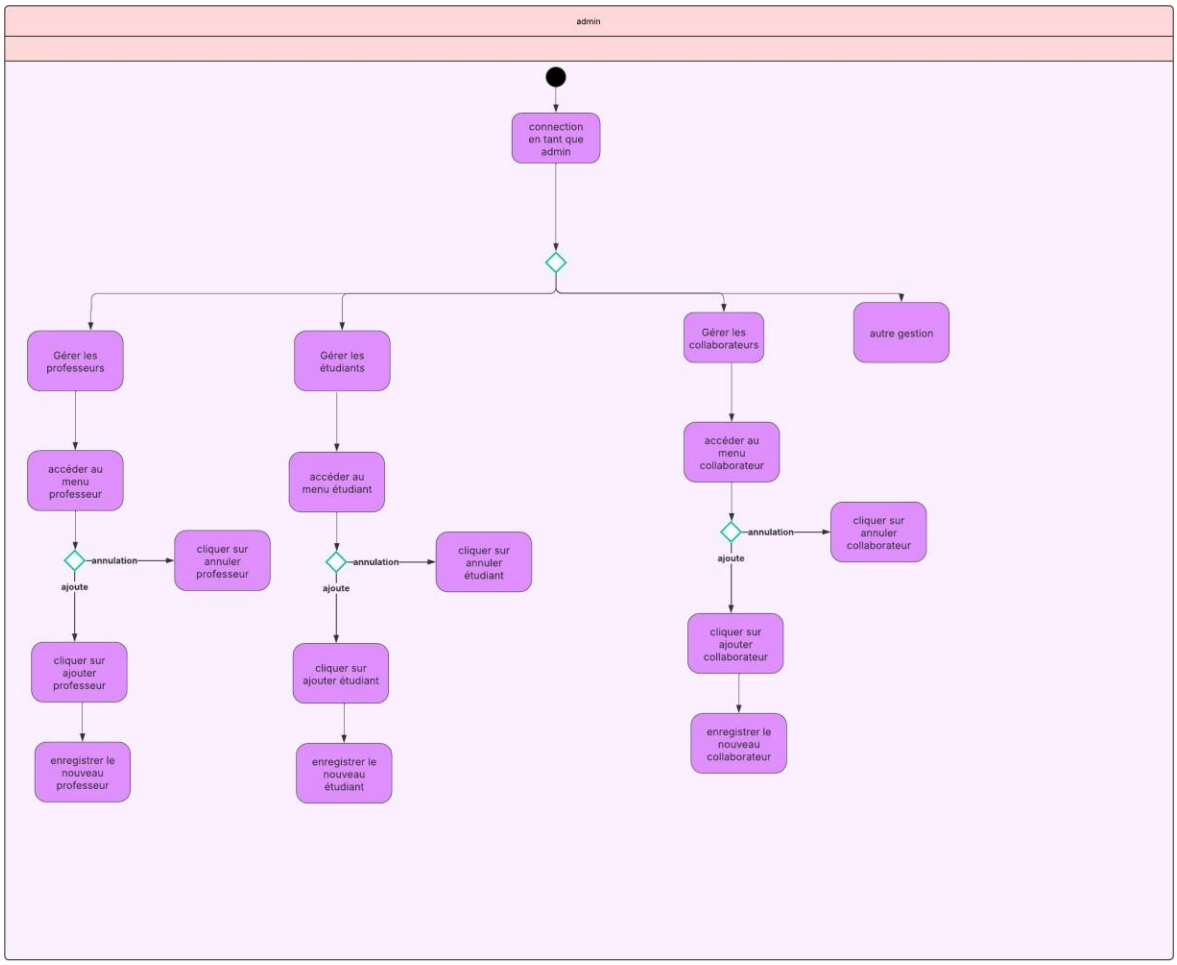


Figure 6 : Diagramme d'activité

Ce diagramme d'activité présente une structure hiérarchique décrivant les fonctionnalités d'un système de gestion éducative ou administrative.

Il est organisé en sections principales, chacune détaillant des actions spécifiques liées à la gestion des professeurs, des étudiants, des collaborateurs, et d'autres tâches annexes.

Par exemple, pour gérer les professeurs, le système permet d'accéder à un menu dédié, d'annuler ou d'ajouter un professeur, et d'enregistrer les modifications.

Une logique similaire s'applique aux étudiants et aux collaborateurs, avec des options comme l'ajout, l'annulation, et l'enregistrement.

Conclusion

Dans ce chapitre, nous avons donné un aperçu sur la démarche conceptuelle que nous avons adoptée pour la réalisation de notre solution. Dans un premier temps nous avons

exploré, dans une conception préliminaire, l'architecture globale de l'application pour passer à le détailler. Ensuite nous avons abordé de manière précise les relations entre les différents composant du système et leurs interactions et ce à l'aide des différents diagrammes de classes et des diagrammes de séquences.

Nous aborderons dans le chapitre suivant la partie réalisation où nous mettons en valeur le travail réalisé.

Chapitre 3

MISE EN OEUVRE

Introduction

Ce chapitre constitue le dernier volet de ce rapport, il traduit la phase qui a pour objectif l'implémentation de notre application. Nous commençons, tout d'abord, par la description de l'environnement matériel et logiciel utilisé pour développer cette application, ensuite nous présentons nos choix technologiques utilisés et enfin nous donnons un aperçu sur le travail réalisé.

I. Environnement de développement

Nous présenterons dans ce qui suit les choix techniques qui nous ont aidés à élaborer notre solution, le matériel utilisé lors de la réalisation de l'application et les environnements logiciels avec lesquels nous avons assuré le développement.

1. Environnement matériel

Le développement de l'application nécessite l'existence d'un environnement matériel spécifique qui est composé de :

• Ordinateur dont les caractéristiques sont comme suit :

Marque : DELL

Mémoire : 16 Go de RAM

Disque Dur : 500 Go

Système d'exploitation : Windows 11

2. Choix technique

Notre application web de gestion des emplois du temps repose sur une architecture client-serveur et suit le modèle de développement MVT (Modèle-Vue-Templates) propre au framework Django.

Pour la partie front-end, nous avons utilisé les technologies HTML5, CSS3 et JavaScript afin d'assurer une interface utilisateur moderne, réactive et adaptée à tous les types de supports (ordinateurs, tablettes, mobiles).

Le back-end a été développé en Python à l'aide du framework Django, qui facilite la gestion de la logique métier ainsi que l'interaction avec la base de données grâce à son ORM intégré.

Concernant la gestion des données, nous avons opté MySQL en production pour sa robustesse et ses performances. La base de données a été soigneusement structurée afin de prendre en charge les différentes entités du système (utilisateurs, emplois du temps, cours, etc.), tout en respectant les exigences de sécurité et d'efficacité. Cette approche modulaire et bien organisée nous a permis de garantir une application maintenable, évolutive et performante.

II. Aperçu sur le travail réalisé

Les interfaces Homme/Machine constituent un élément dans la réussite d'une application.

Ainsi, nous essayerons dans cette partie de présenter les interfaces les plus significatives de notre application.

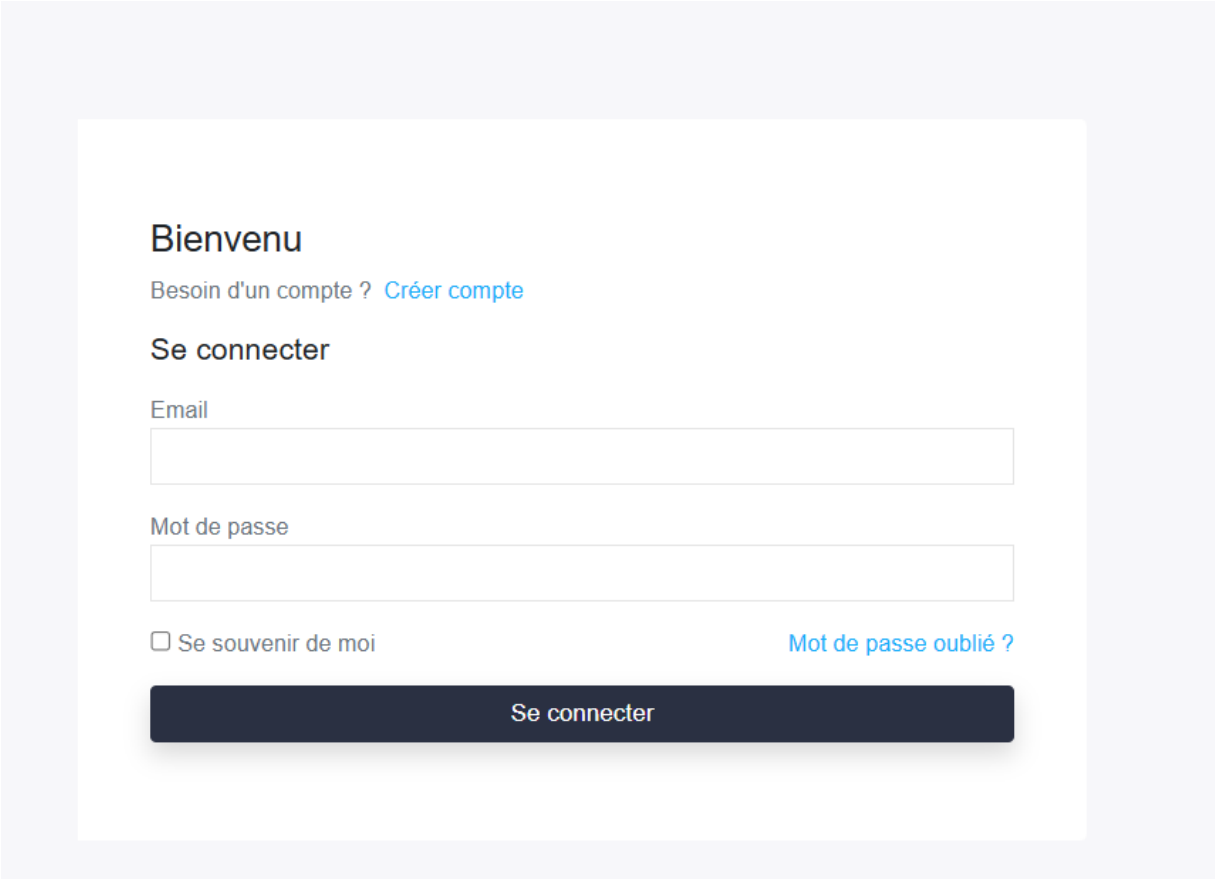


Figure 7 : authentification pour tous les utilisateurs

1. Compte administrateur :

L'admin est invité à lire une description de l'application dans la page d'accueil.

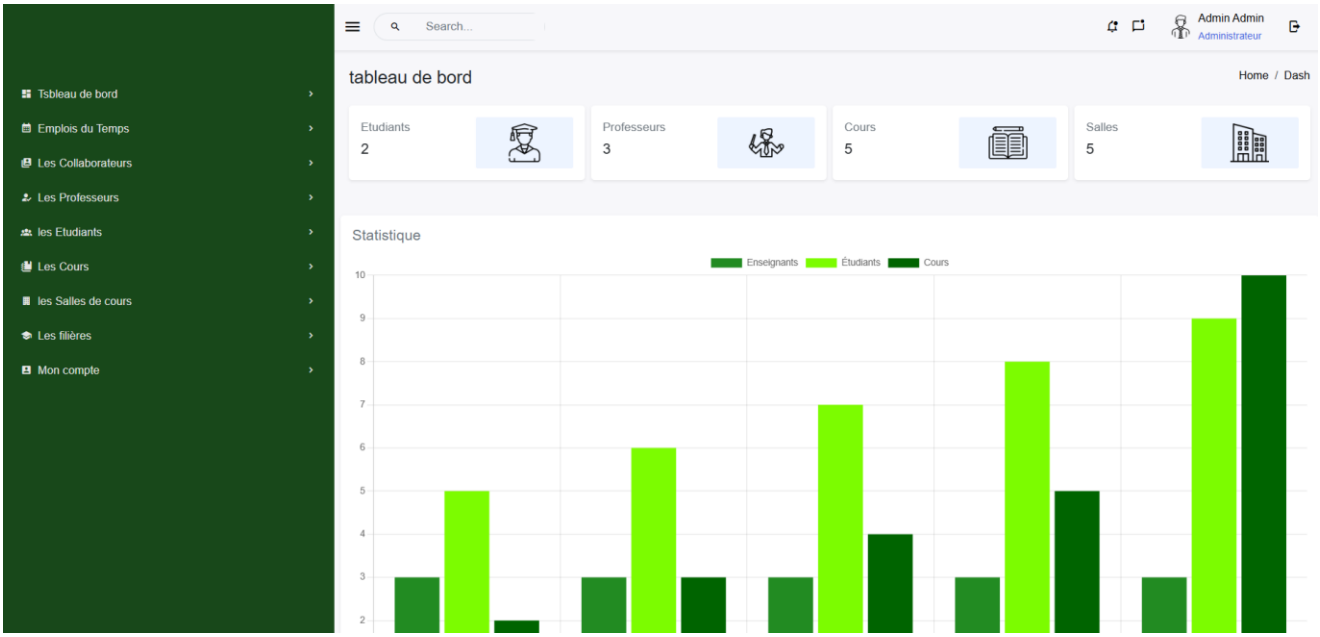


Figure 8 : interface d'accueil

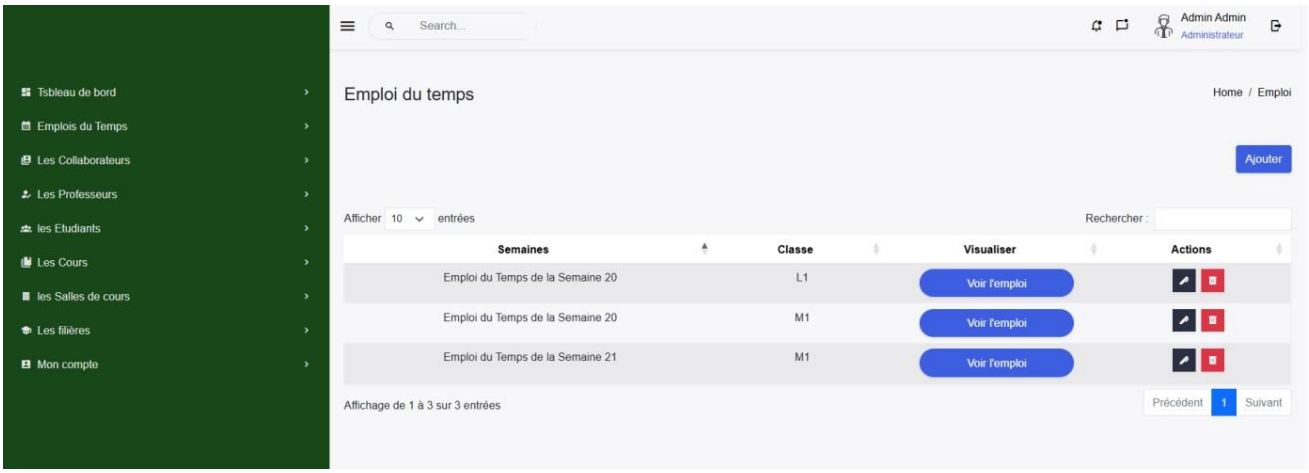


Figure 9 : interface création des emplois du temps

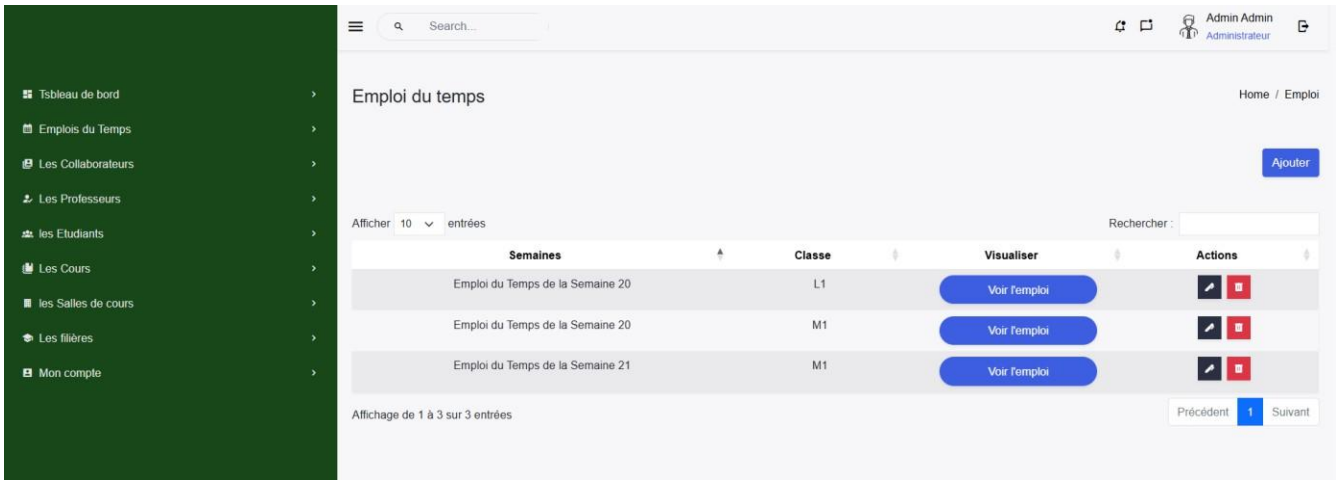


Figure 10 : interface pour l'ajoute des profs

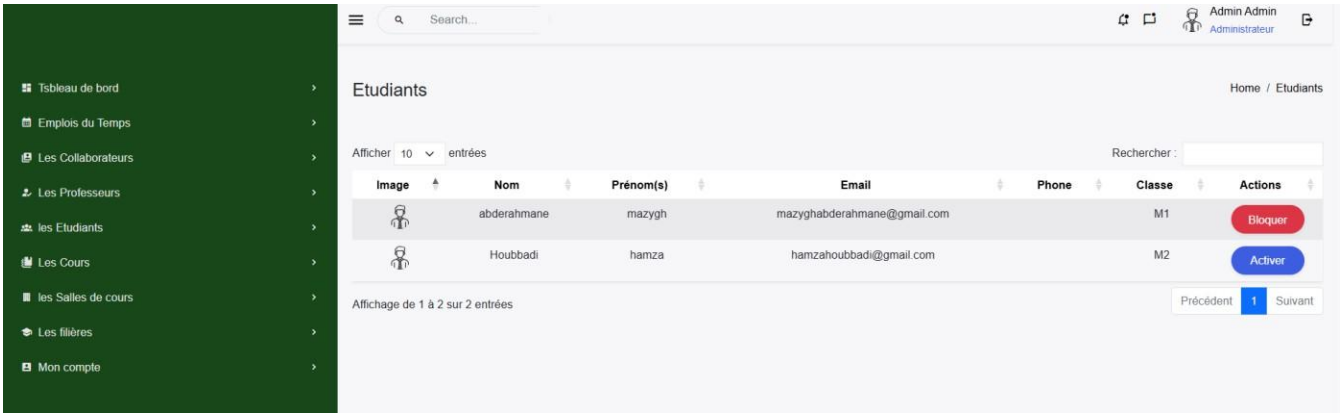


Figure 11 : interface pour l'ajoute des étudiants

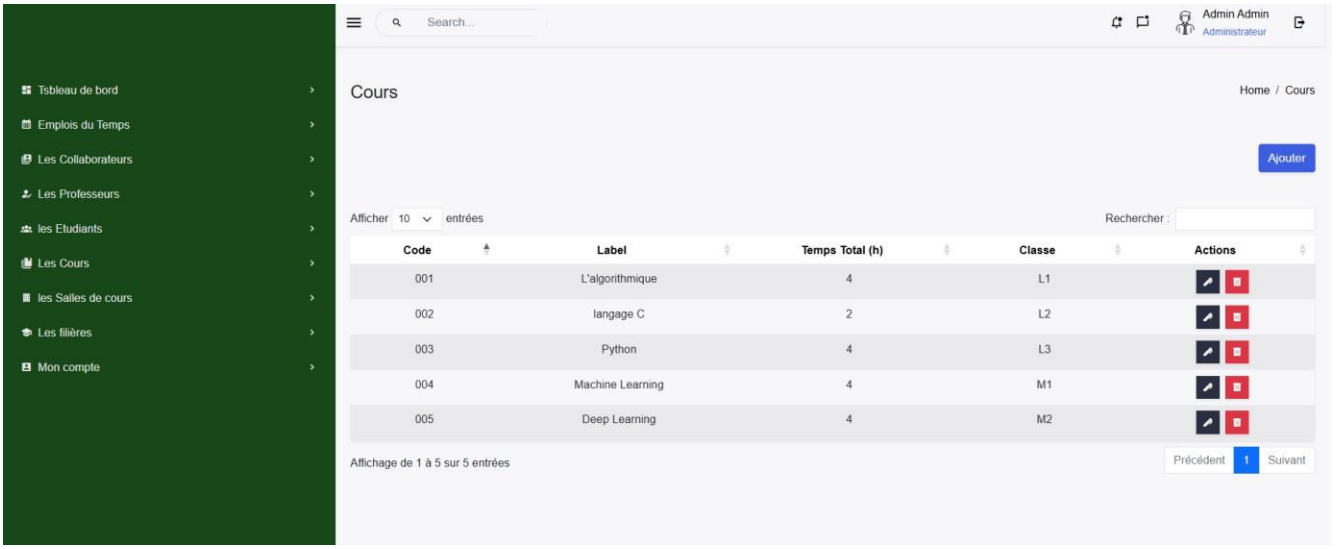


Figure 12 : interface pour l'ajoute des cours

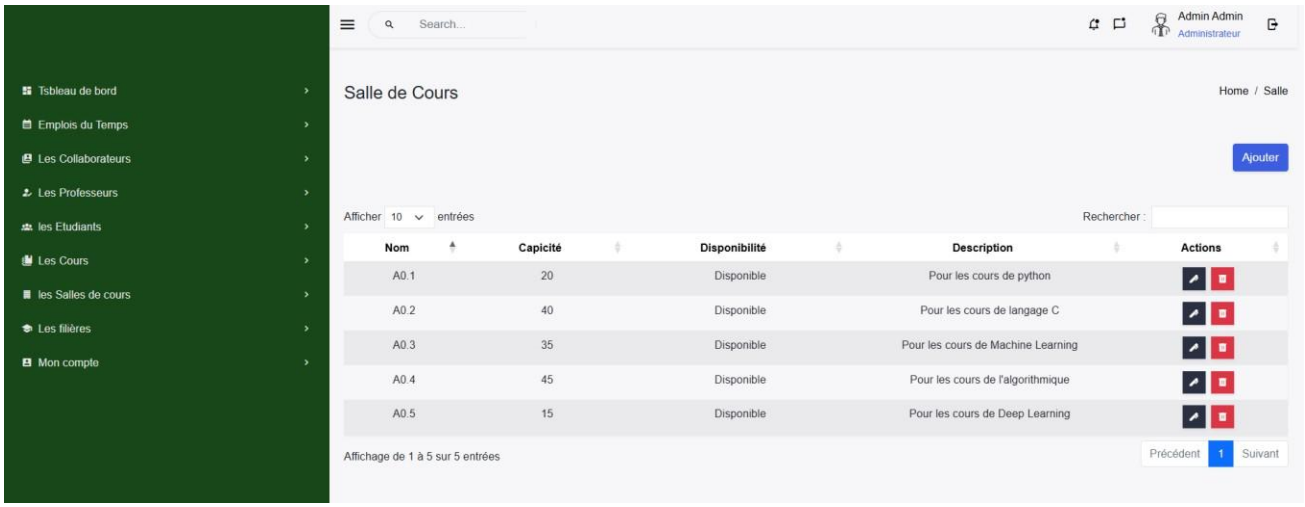


Figure 13 : interface pour l'ajoute des salles cours

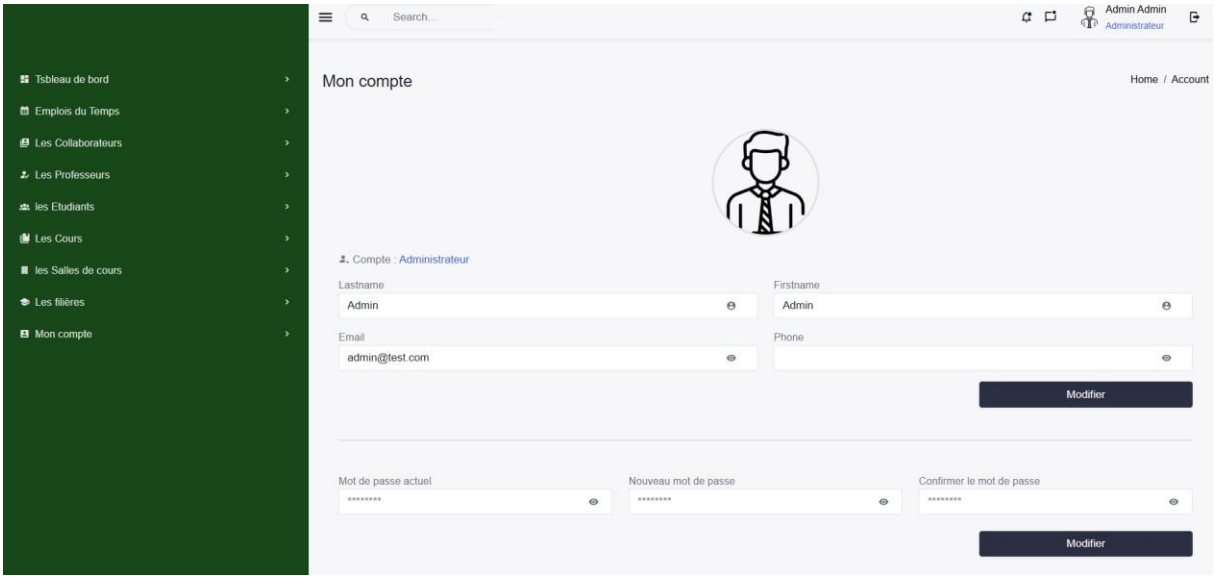


Figure 14 : interface pour profile de l'admin

2. Compte étudiant :

L'étudiant est invité à consulter son compte et fait quelques fonctionnalités une.

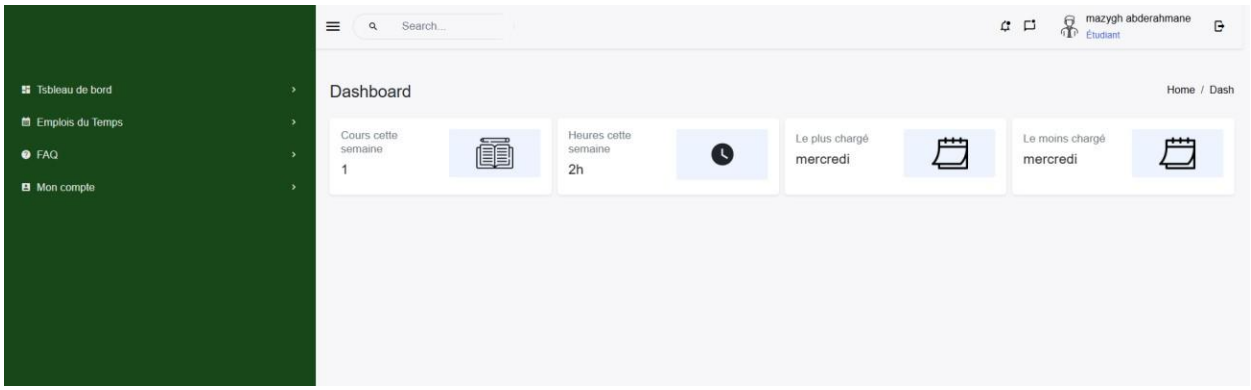


Figure 15 : interface d'accueil

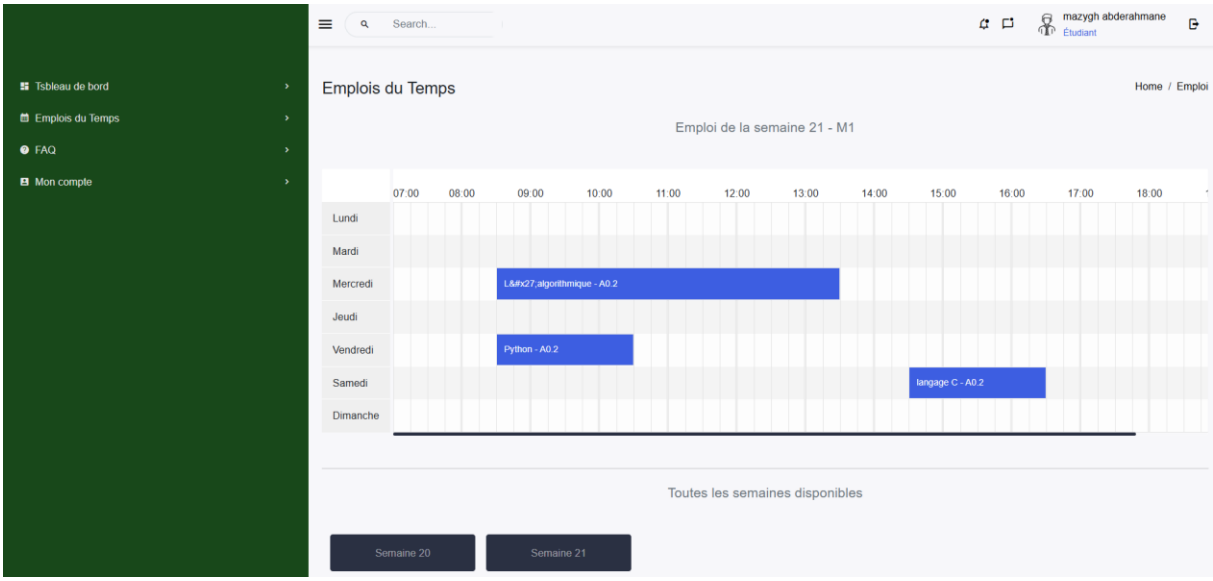


Figure 16 : interface pour consulter emploi du temps

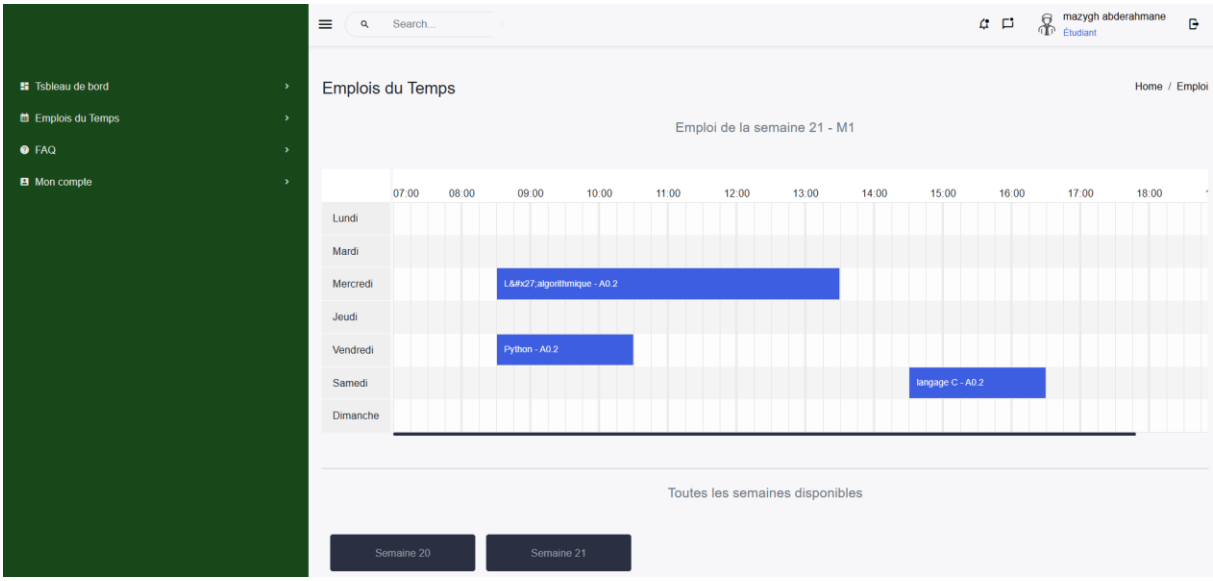


Figure 17 : interface pour le compte de l'étudiant

3. Les sous interfaces :

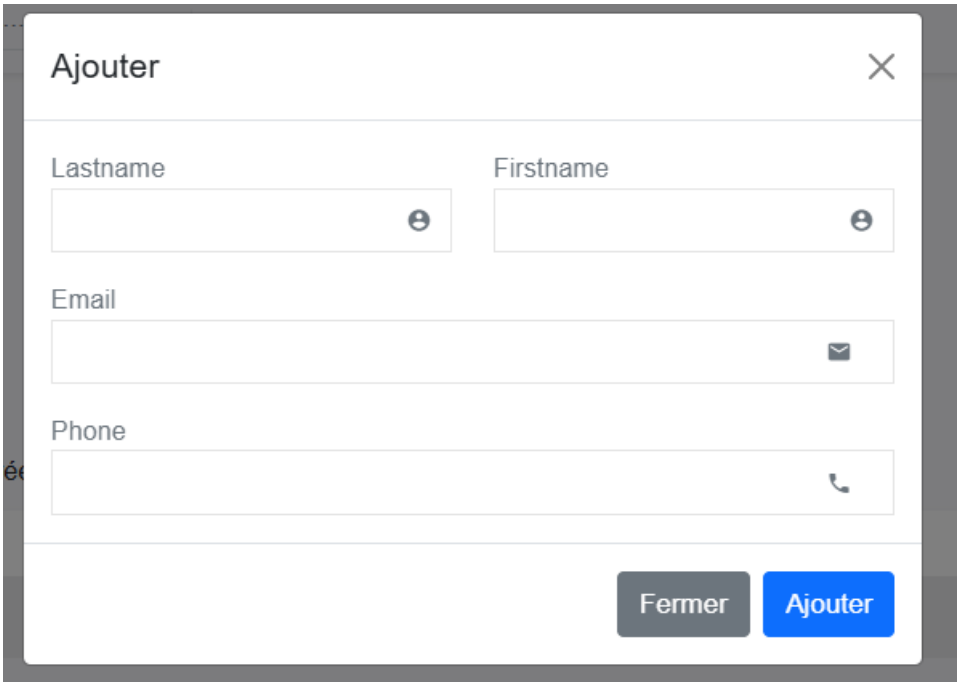


Figure 18 : interface d'ajouter enseignant

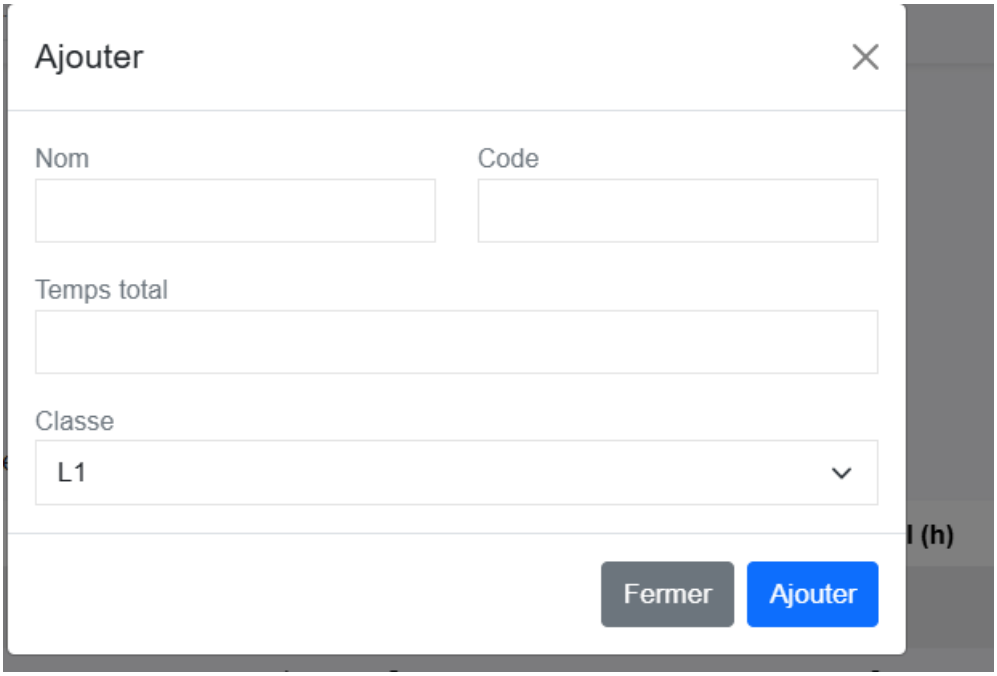


Figure 19 : interface d'ajouter cours

Conclusion

Au cours de ce chapitre, nous avons décrit les plateformes matérielles et logicielles sur lesquelles nous avons construit notre application. Nous avons, ensuite, présenté les interfaces les plus significatives de notre application.

Conclusion Générale

Le domaine des développements connait depuis des années une explosion dans les fonctionnalités matérielles et les services offerts. En mois d'une décennie, les applications ont envahi le monde des universités et sont devenus indispensable dans le management de l'université et des ressources en permettant en même temps la performance, la mobilité et la simplicité.

C'est dans ce cadre que s'inscrit notre projet de fin d'études au sein de l'FP qui consiste à étudier et réaliser une solution de gestion des Emploi de temps. Afin de mener ce travail à bien, nous avons commencé notre projet par une phase de documentation assez approfondie sur la gestion de emploi de temps, des étudiants, des enseignants, ce qui a été le thème du premier chapitre de ce rapport. Nous avons consacré la partie suivante pour l'étude des besoins de l'FP de BM pour bien comprendre les fonctionnalités attendues de notre système. Cette étude a été couronnée par une conception détaillée de la solution faisant sujet du deuxième chapitre et se basant sur les diagrammes UML, à la lumière de laquelle la phase d'implémentation, objet du dernier chapitre, a été mise en œuvre.

Cependant, la nature ouverte du sujet traité laisse plusieurs perspectives ouvertes pour l'améliorer et l'enrichir par des nouvelles fonctionnalités, la simplification de son interface, l'amélioration de la sécurité.

Bibliographie & Webographie

[**B1**] Modélisation objet avec UML : auteurs Pierre-Alain MLLEER & Nathalie GAERTNER

[**B2**] UML Modélisation des objets : auteurs Laurent DEBRAUWER & Fien VAN HEYDE

[N1] http://www.edt-soft.com/

[N2] http://fr.openclassrooms.com/mooc/apprenez-a-creer-votre-site-web-avec-html5-et-css3

[N3] http://www.memoireonline.com/sommaires/stage.html

[N4] http://baudet.me/2013/05/installer-pear-sur-son-wamp/

Guide pour tester :

Assurez-vous d'avoir le language Python installé sur votre machine.

Ouvrez votre terminal et exécutez la commande suivante pour installer Django à l'aide de l'outil de gestion de packages pip :

pip install Django

Si vous n'avez pas pip installé, utilisez plutôt la commande suivante pour installer Django :

python -m pip install Django

Attendez que l'installation se termine. Une fois terminée, vous devriez avoir Django installé sur votre machine.

Pour vérifier si Django est installé correctement, exécutez la commande suivante dans votre terminal :

django-admin --version

Cela affichera la version de Django installée
Assurez-vous d'avoir une version récente de MySQL ici :

Vous devez d'abord crée une base de données sur mysql avec la commande :

CREATE DATABASE nom_de_la_bd;

Une fois la base de données est crée, vous pouvez importer notre base de données au format sql qui se trouve à la racine du projet : pil_database.sql, directement dans la base de données qui vient d'être créer.

Une fois terminer, n'oubliez pas d'ajuster les informations de connection (nom_de_la_bd, utilisateur, mot_de_passe, ...) dans la variable DATABASES.

Installer le pilote mysql pour python.

pip install mysqlclient ou python -m pip install mysqlclient

Nous avons créer par défaut un utilisateur avec le rôle de collaborateur (administrateur). Ce qui vous permettra de vous connecter et tester l'application en tant que admin. Les informations de connexions :

Email : admin@test.com

Mot de passe : adminn

Maintenant nous pouvons lancer l'application avec la commande :

python manage.py runserver 8000

L'application va démarrer sur l'addresse http://localhost:8000 : copier cet addresse et lancer-le dans le navigateur. L'application est maintenant lancé !